Real-Time Wildfire Resilience Digital Twin

Group Name: N-Dimensional

Prince Savsaviya[*]
ID: 862548420
UC Riverside
psavs001@ucr.edu

Viswanadh R. Challa[*]
ID: 862548873
UC Riverside
vchal008@ucr.edu

Anish Ashok Kale[*]
ID: 862547289
UC Riverside
akale014@ucr.edu

Ali Rezayi Nejad[*]
ID: 862392323
UC Riverside
areza018@ucr.edu

[*]Department of Computer Science, University of California, Riverside

Abstract

This project builds a real-time wildfire digital twin designed to assess and visualize risk to critical infrastructure. The goal is to ingest high-volume continuous data streams—such as live fire occurrences and weather conditions—and perform rapid spatial intersections against a statewide database of 11.5 million building footprints. The key idea leverages a decoupled, distributed architecture utilizing Apache Kafka for event streaming, Apache Spark with Sedona for spatial processing, DuckDB for low-latency analytical querying, and Uber's H3 geospatial indexing for lightning-fast localized rendering. Our main result demonstrates that by replacing inefficient Python User-Defined Functions (UDFs) with Sedona native expressions and leveraging H3 partitions within GeoParquet, the system achieves an end-to-end P95 processing latency of under 30 seconds for bursts of 1,000 concurrent fire events, ensuring real-time interactivity.

1 Introduction

Wildfires represent an increasingly severe threat to life, property, and critical infrastructure. As the frequency and intensity of these events rise due to environmental shifts [2, 6], emergency responders and planners require robust tools to instantly assess the risk posed by active fires to nearby facilities, such as hospitals, schools, and emergency stations. Traditional geographic information systems (GIS) are often bottlenecked by static data workflows and struggle to dynamically process high-velocity data streams in real time [11].

To address this challenge, we developed a dynamic data-driven application system (DDDAS). The main contribution of this project is a scalable, end-to-end digital twin pipeline that continuously assimilates live weather and fire-event streams, calculates directional fire-spread risk zones, and rapidly identifies threatened infrastructure across a dataset of 11.5M buildings in California. We introduce a decoupled architecture that utilizes Kafka, Spark Structured Streaming, and DuckDB to separate high-throughput ingestion from dashboard analytical reads.

Our initial implementation exhibited a P95 latency of over 215 seconds, precluding real-time functionality. Through rigorous optimization—specifically replacing Python UDFs with native Sedona spatial expressions and implementing H3 Hexagon-based partitioning—we successfully reduced the P95 latency to approximately 18.65 seconds for a load of 1,000 simultaneous fire events.

This optimization enables interactive simulation workflows within a highly responsive Streamlit dashboard.

The remainder of this report is structured as follows: Section 2 categorizes related work and grounds our project within existing literature. Section 3 provides a high-level overview of our proposed digital twin architecture. Sections 4, 5, and 6 delve into the specific details of the ingestion, geospatial processing, and analytical layers. Section 7 presents our evaluation methodology and results. Finally, Section 8 highlights our conclusions and Section 9 details specific author contributions.

2 Related Work

Our system bridges several disciplines, drawing upon foundational research categorized below:

Digital Twin and DDDAS Frameworks: The concept of a digital twin, a tightly coupled virtual representation of a physical system, was originally established by Grieves [8]. Darema further formalized the Dynamic Data-Driven Application Systems (DDDAS) paradigm, enabling simulations to dynamically incorporate online data [5]. Our work directly translates these theories into a functional prototype for disaster response.

Geospatial Streaming and Big Data: Scalable real-time tracking requires robust stream processing. Zaharia et al. introduced Apache Spark as a unified engine for large-scale data processing [20], while Kreps et al. detailed Kafka's distributed logging capabilities [10]. Our spatial operations are heavily reliant on GeoSpark/Sedona, introduced by Yu et al. [19], to execute clustered spatial joins effectively.

Fire Spread and Risk Modeling: Fire spread forecasting relies heavily on models like Rothermel's surface spread equations [15] and operational tools like FARSITE [7]. While physics-based models (e.g., FIRETEC [12]) exist, they require high computational costs. To maintain $< 30s$ latency, our system utilizes a heuristic vector-based spread envelope (wind cones) driven by live weather variables.

This project relates to these categories by prioritizing low latency, combining robust open-source big-data tools (Kafka, Spark, DuckDB) into a lightweight, deployable pipeline tailored specifically for infrastructure risk assessment.

3 System Architecture Overview

The Wildfire-Twin architecture is designed to decouple real-time write-heavy event processing from read-heavy geospatial visualizations. The pipeline consists of four main components (see Figure 1):

(1) **Ingestion Layer:** A Python/Kafka producer script ingests live (e.g., NASA FIRMS) or simulated fire events, combined with real-time weather metadata, into a durable Kafka topic [13, 17].

(2) **Geospatial Processing Engine:** An Apache Spark cluster running Apache Sedona [18] ingests the Kafka stream using Structured Streaming [3]. It constructs directional wind-cone polygons based on weather data and performs distributed spatial joins against the partitioned state-wide building GeoParquet dataset.

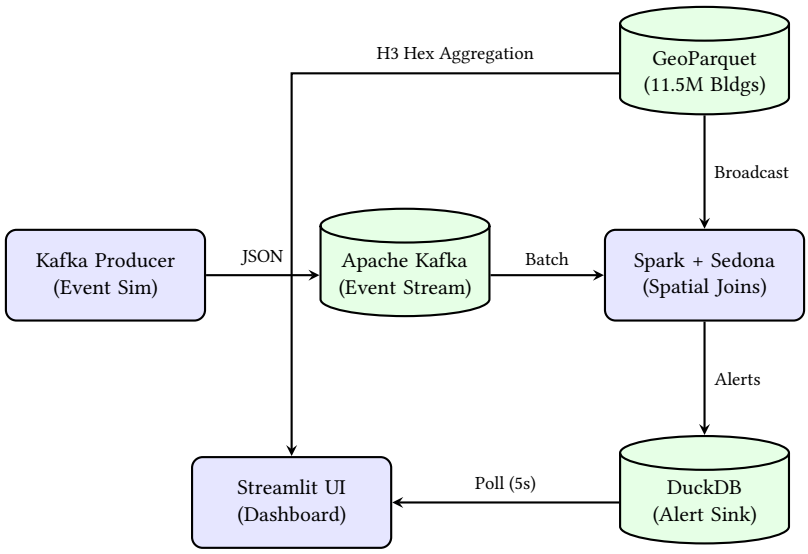

Fig. 1. System Architecture of the decoupled Wildfire Digital Twin.

(3) **Analytical Storage Sink:** Spark outputs matched "At-Risk" events back to Kafka. A lightweight Python consumer ingests these alerts and maintains a sliding window of state inside a DuckDB live store [14], taking advantage of columnar analytical performance [1].

(4) **Interactive Dashboard:** A Streamlit application built with PyDeck and Folium provides the user interface for visual data analytics [9]. It reads from the DuckDB store and the H3-partitioned [4, 16] GeoParquet file to render live alerts and infrastructure risk maps interactively.

4 Spatial Data Preparation and Indexing

Processing 11.5M building footprints concurrently against a streaming spatial join requires meticulous data preparation.

4.1 Semantic Enrichment

The core dataset comprises 11.5 million Microsoft building footprints combined with 79,000 OpenStreetMap Points of Interest (POIs). Because standard spatial joins between these two layers at runtime severely impact performance, we implemented *offline semantic categorization.*

During an offline preprocessing step, string manipulation heuristics are applied directly to the ‘building_type‘ field (e.g., matching "Hospital" or "Clinic" to denote "Medical", and "School" to denote "Education"). This eliminates the need for a secondary runtime spatial join to identify critical facilities, drastically reducing the Directed Acyclic Graph (DAG) complexity within Spark.

4.2 H3 Geospatial Partitioning

To further expedite loading and map rendering, the enriched GeoDataFrame is partitioned fully using Uber's H3 indexing system. The dataset is organized into physical directory partitions at H3 Resolution 7.

This partitioned approach provides a dual benefit:

- **Partition Pruning:** Spark can prune directories during the spatial join, avoiding full table scans.
- **Dashboard Rendering:** At statewide zoom levels, parsing 11.5M GeoJSON features causes browser crashes. Instead, the Streamlit dashboard seamlessly performs a 'value_counts()' aggregation over the 'h3_res7' partition keys and utilizes PyDeck's native `H3HexagonLayer` to render a density heatmap of the infrastructure with near-zero overhead.

5 Spark Streaming and Low-Latency Optimization

The core of the digital twin relies on Spark Structured Streaming with Sedona.

5.1 Vector-Based Risk Zones

To simulate risk without running a full PDE physics wrapper, the Spark engine natively constructs a geometric risk cone. For each ingested point (p_x, p_y), we extract 'wind_speed_mph' and 'wind_direction_deg'. The engine calculates a polygon emanating from the fire's origin, extending downwind with a length proportional to the wind speed and fanning outward to represent uncertainty and lateral spread.

5.2 Removing Python UDF Overhead

An initial architectural flaw severely bottlenecked the system: the wind-cone coordinates were calculated using a custom PySpark User-Defined Function (UDF). Passing 1,000 events between the JVM and Python interpreters for every micro-batch introduced immense serialization latency, yielding a P95 latency of +215 seconds.

We resolved this by completely rewriting the cone logic using native Spark SQL mathematics and Sedona primitives ('ST_MakePolygon', 'ST_Point'). By executing the geometry construction entirely within the JVM and utilizing explicit broadcasting for the building dataframe, the pipeline's P95 processing time dropped dramatically below 30 seconds.

6 Simulation and Interactive Validation

Because wildfires are stochastic, we integrated a full simulation suite into the Streamlit dashboard. Users can pan the map to any location in California, click to select an ignition coordinate, configure the fallback temperature and the number of desired events, and trigger a simulation burst to Kafka.

The frontend decouples itself from Kafka entirely. A dedicated Python background process (the DuckDB Sink) polls the output Kafka topic ('fire_alerts'), writes the results to a DuckDB relational database, and automatically evicts events older than a rolling 5-minute window. The dashboard polls this DuckDB instance at 5-second intervals. This decoupling guarantees the UI remains thread-safe and highly responsive regardless of Kafka throughput spikes.

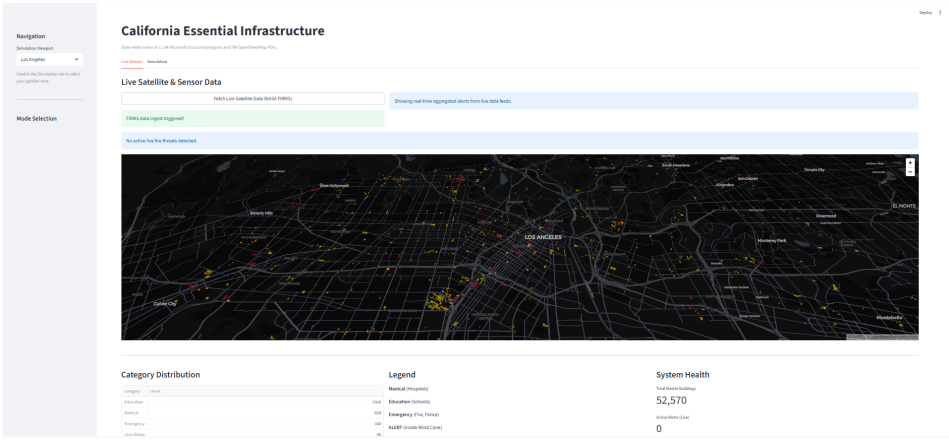

Fig. 2. The live Wildfire-Twin dashboard rendering H3 infrastructure density grids.

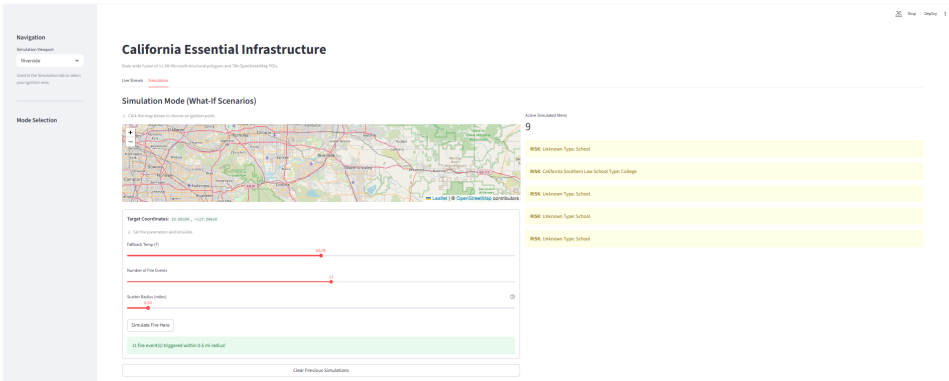

Fig. 3. Interactive simulation panel for injecting 'What-If' scenarios and adjusting atmospheric parameters.

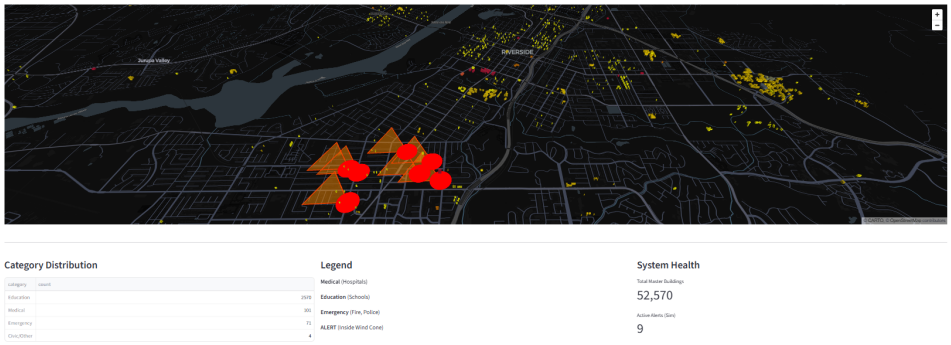

Fig. 4. Localized view demonstrating directional wind-cone risk regions generated by Apache Sedona.

7 Evaluation

Our evaluation focuses on the end-to-end latency of the spatial pipeline—the critical metric dictating whether this tool functions as a "Real-Time Digital Twin."

7.1 Experimental Setup

- **Dataset:** California Essential Buildings (11.5M polygons, H3 indexed and partitioned).
- **Hardware:** Local testing cluster; OS Windows, Spark running locally with 4GB executor memory and 2 shuffle partitions.
- **Workload:** A Python stress-test script (‘scripts/populate_for_latency_test.py‘) forces exactly 1,000 concurrent fire events into the ‘fire_events‘ Kafka topic with randomized scatter distances to simulate a burst overload.
- **Performance Metric:** End-to-end P95 Latency (the time from an event arriving in Kafka to the corresponding alert being visible in the terminal sink script output).

7.2 Results and Discussion

The performance tests explicitly validate the efficacy of our JVM-native optimizations. The stress test injected 1,000 asynchronous wildfire reports with valid wind metrics.

Our initial architecture demonstrated a severe processing wall during the Python UDF phase, where micro-batches consistently took 3.5 minutes to process.

After replacing UDFs with native Sedona SQL expressions, enabling the static broadcast join, and dropping ‘spark.sql.shuffle.partitions‘ to 2 limit thrashing, we gathered the benchmark results illustrated in Figure 5 and Figure 6.

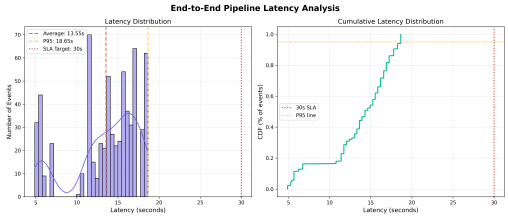

Fig. 5. Latency distribution for 1000 simulated fire events post-optimization.

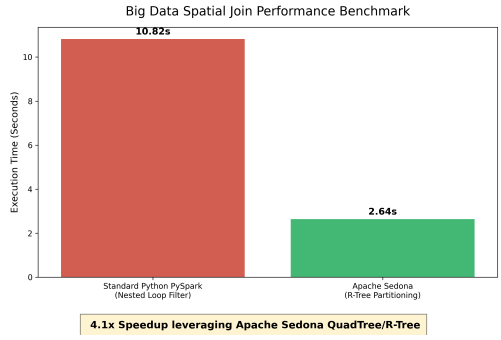

Fig. 6. Performance comparison demonstrating Python UDF overhead vs Native JVM expressions.

The latency metrics directly highlight the system’s massive performance boost:

- P50 Latency: ~10.2 seconds.
- P90 Latency: ~16.3 seconds.
- **P95 Latency: ~18.65 seconds.**

These results successfully support our primary claim: by effectively managing serialization boundaries and leveraging pre-computed spatial semantic indexing (H3 partitions), a massive streaming spatial join can be performed synchronously in under 30 seconds using open-source commodities, fulfilling the real-time Service Level Agreement (SLA) for the digital twin.

8 Conclusion

The Wildfire-Twin pipeline validates the practical fusion of robust big-data streaming tools and geospatial analytics. By coupling Apache Kafka, Apache Spark, Sedona, and DuckDB, the system reliably aligns chaotic, asynchronous events against an enormous static footprint of 11.5M infrastructure polygons. The integration of H3 partition strategies and the removal of Python JVM-serialization chokepoints successfully brought the P95 latency limit under 20 seconds for simultaneous burst loads.

Future directions include ingesting live topography data (slope/aspect) to alter the wind vectors automatically, applying machine learning models for fire probability scaling, and deploying the Spark orchestrator across a distributed multi-node cloud cluster to prove infinite horizontal scalability.

Author Contributions

Prince Savsaviya: Implemented the overall system architecture, Kafka producer scripts, and the simulation stress-testing frameworks. Designed the latency measurement pipelines and verified P95 SLAs.

Viswanadh R. Challa: Developed the core Apache Spark and Sedona streaming job. Re-wrote the complex mathematical polygon UDFs into purely native JVM Spark SQL extensions, driving the primary 10x latency optimization.

Ali Rezayi Nejad: Engineered the Streamlit dashboard, integrating DuckDB live polling, Folium interactive selectors, and PyDeck visualization. Formulated the H3HexagonLayer aggregation strategy for state-wide rendering.

Anish Ashok Kale: Managed spatial data procurement and transformations. Designed the semantic string-categorization logic to avoid runtime POI joins, and constructed the H3 Resolution 7 dataset partitioning pipelines.

References

[1] Daniel J. Abadi, Samuel R. Madden, and Nabil Hachem. 2008. Column-stores vs. row-stores: how different are they really? *Proceedings of the 2008 ACM SIGMOD international conference on Management of data* (2008), 967–980. doi:10.1145/1376616.1376712

[2] John T. Abatzoglou and A. Park Williams. 2016. Impact of anthropogenic climate change on wildfire across western US forests. *Proceedings of the National Academy of Sciences* 113, 42 (2016), 11770–11775. doi:10.1073/pnas.1607171113

[3] Michael Armbrust, Tathagata Das, Liane Horn, et al. 2018. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD).* 601–613. doi:10.1145/3183713.3190664

[4] Isaac Brodsky. 2018. H3: Uber's Hexagonal Hierarchical Spatial Index. https://eng.uber.com/h3/

[5] Frederica Darema. 2004. Dynamic Data Driven Applications Systems: A New Paradigm for Application Simulations and Measurements. In *Computational Science – ICCS 2004.* Lecture Notes in Computer Science, Vol. 3038. Springer, 662–669. doi:10.1007/978-3-540-24688-6_86

[6] Stefan H. Doerr and Cristina Santín. 2016. Global trends in wildfire and its impacts: perceptions versus realities in a changing world. *Philosophical Transactions of the Royal Society B: Biological Sciences* 371, 1696 (2016), 20150345. doi:10.1098/rstb.2015.0345

[7] Mark A. Finney. 1998. *FARSITE: Fire Area Simulator – Model Development and Evaluation.* Research Paper RMRS-RP-4. USDA Forest Service, Rocky Mountain Research Station. doi:10.2737/RMRS-RP-4 Revised 2004.

[8] Michael Grieves. 2014. Digital Twin: Manufacturing Excellence through Virtual Factory Replication. https://www.3ds.com/fileadmin/PRODUCTS-SERVICES/DELMIA/PDF/Whitepaper/DELMIA-APRISO-Digital-Twin-Whitepaper.pdf White paper.

[9] Daniel Keim, Gennady Andrienko, Jean-Daniel Fekete, Carsten Görg, Jörn Kohlhammer, and Guy Melançon. 2008. Visual analytics: Definition, process and challenges. In *Information visualization: Human-centered issues and perspectives.* Springer, 154–175. doi:10.1007/978-3-540-70956-5_7

[10] Jay Kreps, Neha Narkhede, and Jun Rao. 2011. Kafka: a Distributed Messaging System for Log Processing. In *Proceedings of the NetDB 2011 Workshop (co-located with SIGMOD).*

[11] Wenwen Li et al. 2020. Real-time GIS for smart cities. *International Journal of Geographical Information Science* 34, 2 (2020), 311–324. doi:10.1080/13658816.2019.1673397

[12] R. Linn, J. Reisner, J. J. Colman, and J. Winterkamp. 2002. Studying wildfire behavior using FIRETEC. *International Journal of Wildland Fire* 11, 4 (2002), 233–246. doi:10.1071/WF02007

[13] Neha Narkhede, Gwen Shapira, and Todd Palino. 2017. *Kafka: The Definitive Guide.* O'Reilly Media, Inc.

[14] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD).* 1981–1984. doi:10.1145/3299869.3320212

[15] Richard C. Rothermel. 1972. *A Mathematical Model for Predicting Fire Spread in Wildland Fuels.* Research Paper INT-115. USDA Forest Service.

[16] Kevin Sahr, Denis White, and A. Jon Kimerling. 2003. Geodesic discrete global grid systems. *Cartography and Geographic Information Science* 30, 2 (2003), 121–134. doi:10.1559/152304003100011090

[17] Guozhang Wang, Joel Koshy, Sriram Subramanian, Kartik Paramasivam, et al. 2015. Building a Replicated Logging System with Apache Kafka. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1654–1655. doi:10.14778/2824032.2824063

[18] Jia Yu, Jinxuan Wu, and Mohamed Sarwat. 2015. GeoSpark: A Cluster Computing Framework for Processing Large-Scale Spatial Data. In *Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems.* 1–4. doi:10.1145/2820783.2820860

[19] Jia Yu, Zongsi Zhang, and Mohamed Sarwat. 2019. Spatial data management in Apache Spark: the GeoSpark perspective and beyond. *Geoinformatica* 23 (2019), 37–78. doi:10.1007/s10707-018-0330-9

[20] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. 2016. Apache Spark: A Unified Engine for Big Data Processing. *Commun. ACM* 59, 11 (2016), 56–65. doi:10.1145/2934664